

Rat-Mice: Ergodicity

Stella SRZICH (3371930)

July 30, 2026

This document is meant to serve as a discussion of the Rat-Mice algorithm and sketch of the proof of its ergodicity.

1 What kind of algorithm do we want?

In general, we want an algorithm that correctly samples from a target density $\pi(\cdot)$ (i.e., produces a Markov chain that is ergodic with respect to $\pi(\cdot)$). Moreover, we would like the chain to converge to the target density within a reasonable number of iterations. Moreover, out of all such algorithms, we would like to minimize the computational cost per effective sample.

The standard algorithm that indeed correctly samples from a target density $\pi(\cdot)$ is the Metropolis–Hastings (MH) algorithm.

Algorithm 1 Metropolis–Hastings (MH)

function: $[x^1, \dots, x^N] = \text{MH}(\pi(\cdot), q(\cdot | \cdot), x^0, N)$

input: target density $\pi(\cdot)$, proposal density $q(\cdot | \cdot)$, initial state x^0 , number of steps N

output: ordered list of states $[x^1, \dots, x^N]$

- 1: **for** $j = 0$ to $N - 1$ **do**
- 2: Given x^j , generate a proposal y distributed as $q(y | x^j)$.
- 3: Accept proposal y as the next state, i.e. set $x^{j+1} = y$, with probability

$$\alpha(y | x^j) = \min \left\{ 1, \frac{\pi(y)q(x^j | y)}{\pi(x^j)q(y | x^j)} \right\}.$$

- 4: Otherwise reject y and set $x^{j+1} = x^j$.
 - 5: **end for**
-

However, when the target density $\pi(\cdot)$ is multimodal (e.g., as is the case when we use high-level representations for states), with basic choices of proposal kernel $q(\cdot | \cdot)$, the MH algorithm can struggle to explore all modes within a reasonable number of iterations. In this case, the MH algorithm will not converge to the target density within a reasonable number of iterations.

One way to address this is to, at each iteration, run a subchain of length n that targets a coarse (i.e., cheap to evaluate) density $\pi_C(\cdot)$, and then use the resulting state as a proposal for this iteration. The idea is that if the coarse density $\pi_C(\cdot)$ approximates the target density $\pi(\cdot)$ well enough, after a sufficient number of subchain steps, the proposal will be in a different mode. This effectively produces a new proposal kernel (from the basic proposal kernel $q(\cdot | \cdot)$) for use in the MH algorithm that “wormholes” between modes without us having to be clever about explicitly designing such a proposal. Or, as we like to think of it, a proposal kernel of little “rats and

mice” that tunnel between the modes. This idea is implemented in the Randomised-Length-Subchain Surrogate Transition (RST) algorithm.

Algorithm 2 Randomised-Length-Subchain Surrogate Transition (RST)

function: $[x^1, \dots, x^N] = \text{RST}(\pi_F(\cdot), \pi_C(\cdot), q(\cdot | \cdot), p(\cdot), x^0, N)$

input: target (fine) density $\pi_F(\cdot)$, surrogate (coarse) density $\pi_C(\cdot)$, proposal kernel $q(\cdot | \cdot)$, probability mass function $p(\cdot)$ over subchain length, initial state x^0 , number of steps N

output: ordered list of states $[x^1, \dots, x^N]$ (or just the final state x^N)

1: **for** $j = 0$ to $N - 1$ **do**

2: Draw the subchain length $n \sim p(\cdot)$.

3: Starting at x^j , generate a subchain of length n using MH Alg. 1 to target $\pi_C(\cdot)$:

$$y = \text{MH}(\pi_C(\cdot), q(\cdot | \cdot), x^j, n).$$

4: Accept the proposal y as the next sample, i.e. set $x^{j+1} = y$, with probability

$$\alpha(y | x^j) = \min \left\{ 1, \frac{\pi_F(y)\pi_C(x^j)}{\pi_F(x^j)\pi_C(y)} \right\}.$$

5: Otherwise reject y and set $x^{j+1} = x^j$.

6: **end for**

To make the RST algorithm more efficient, one would like to estimate the mean and covariance of the discrepancy between the exact and coarse forward maps and subsequently modify the coarse likelihood so that the resulting coarse density more closely approximates the target density as the chain progresses (see Fox, Cui and Neumayer 2020).

If we allow state-dependent coarse densities $\pi_{C,x}(\cdot)$, as opposed to a single state-independent coarse density $\pi_C(\cdot)$ as in the RST algorithm, the coarse forward map can be locally corrected so that it agrees with the exact forward map at the current state x . For this locally corrected map, the discrepancy is zero at x and has mean zero under the stationary transition law. For some reason (I think because we no longer need to correct the global coarse model error, we only need to correct the change in the coarse model error over one move), the modified state-dependent coarse densities more closely approximate the target density than the modified state-independent coarse density. Long story short, this results in a more efficient algorithm.

We name this algorithm the Rat-Mice (RM) algorithm. For simplicity, we begin by only considering a fixed subchain length n .

Algorithm 3 Rat-Mice (RM)

function: $[x^1, \dots, x^N] = \text{RM}(\pi(\cdot), \{\pi_{C,x}(\cdot)\}_{x \in \mathcal{X}}, q(\cdot | \cdot), n, x^0, N)$

input: target density $\pi(\cdot)$, state-dependent surrogate (coarse) densities $\{\pi_{C,x}(\cdot)\}_{x \in \mathcal{X}}$, proposal kernel $q(\cdot | \cdot)$, subchain length n , initial state x^0 , number of steps N

output: ordered list of states $[x^1, \dots, x^N]$

1: **for** $j = 0$ to $N - 1$ **do**

2: Starting at x^j , generate subchain of length n using MH Alg. 1 to target $\pi_{C,x_j}(\cdot)$:

$$y = \text{MH}(\pi_{C,x_j}(\cdot), q(\cdot | \cdot), x^j, n).$$

3: Accept the state y as our proposal, i.e. set $x^{j+1} = y$, with probability

$$\alpha(y | x^j) = (\text{some value that ensures the chain is in detailed balance with } \pi(\cdot)).$$

4: Otherwise reject y and set $x^{j+1} = x^j$.

5: **end for**

We now recap the point of the RM algorithm. If you use basic proposal kernels (i.e., ones you are not super clever about), the MH algorithm will not converge to a multimodal target density within a reasonable number of iterations. One way to address this is to use the basic proposal kernel to generate a more sophisticated proposal kernel that acts like little “rats and mice” that tunnel between the modes.

To generate this more sophisticated proposal kernel, you want to target a density that is cheap to evaluate (so that doing a bunch of steps in the subchain is cheap), and you want the density to be a good approximation of the target density (so that proposals kind of look like samples from the target density and are accepted with high probability). The RST algorithm does this, but only allows for a single state-independent coarse density.

Given a state-independent coarse density (which will be defined by a coarse forward map), we can easily define state-dependent coarse densities that are better approximations of the target density (and are as cheap to evaluate as they use the same coarse forward map). Thus, we want to somehow modify the RST algorithm to allow for state-dependent coarse densities.